

Software Requirements Specification (SRS)

Web-Based Password Strength Meter and Breached Credential Checker

Version: 1.0

Date: May 2026

Authors: Abubakar Muhammad Sherif, Madu-Uwechie Chukwunonso Noel, Adriel Madukwe Ogbureke, Stefan Birch Olufikayo **Institution:** Nile University of Nigeria

Course: Web Application Development (SEN 302) **Status:** Final

1. Introduction

1.1 Purpose

This Software Requirements Specification describes the functional and non-functional requirements for a WEB-BASED PASSWORD STRENGTH METER AND BREACHED CREDENTIAL CHECKER application. The document is intended to guide the design, development, and testing of the system, and to serve as a reference throughout the Software Development Lifecycle (SDLC).

This document is produced as part of a university software engineering assignment and follows the IEEE 830 standard for software requirements specification.

1.2 Scope

The system, referred to hereafter as **PassGuard**, is a full-stack web application built using the MERN stack (MongoDB, Express.js, React.js, Node.js).

PassGuard enables users to:

- Analyse the strength of a password in real time
- Receive actionable feedback on how to improve password quality
- Check whether a password has been exposed in known data breaches
- Create an account to save and review their breach check history

The application will be deployed as a publicly accessible web application responsive on both desktop and mobile devices.

The system will NOT:

- Store any user-entered passwords in plain text or any readable form
- Transmit raw passwords to any external service
- Provide password management or storage features
- Guarantee protection against all forms of cyber attack

1.3 Definitions and Abbreviations

Term - Definition

MERN - MongoDB, Express.js, React.js, Node.js — a full-stack JavaScript framework

SRS - Software Requirements Specification

SDLC - Software Development Lifecycle

API - Application Programming Interface

JWT - JSON Web Token — used for stateless user authentication

bcrypt - A password hashing algorithm used to store passwords securely

HIBP - Have I Been Pwned — a public API for checking breached credentials

k-Anonymity - A privacy technique that avoids sending full data to external services

SHA-1 - Secure Hash Algorithm 1 — used to hash passwords before breach checking

zxcvbn - A password strength estimation library developed by Dropbox

FR - Functional Requirement

NFR - Non-Functional Requirement

UI - User Interface

UX - User Experience

1.4 Document Overview

The remainder of this document describes the overall system, its users, its functional and non-functional requirements, use cases, architecture, database design, and testing strategy. Sections are organised to follow the natural progression of the Software Development Lifecycle.

2. Overall Description

2.1 Product Perspective

PassGuard is a standalone web application. It interacts with two external services:

1. **Have I Been Pwned API** — This is for checking whether a password hash prefix appears in known breach databases
2. **MongoDB Atlas** — It is a cloud-hosted database for storing user accounts and check history

The system will follow a standard client-server architecture. The React.js frontend communicates with the Node.js/Express.js backend via a RESTful API. The backend communicates with MongoDB Atlas for data persistence and with the HIBP API for breach detection.

2.2 Product Functions

The primary functions of PassGuard are:

1. **Real-time password strength analysis** — scoring passwords as the user types, using the zxcvbn library.

2. **Visual strength feedback** — displaying a colour-coded progress bar and descriptive label (Weak / Fair / Good / Strong / Very Strong).
3. **Actionable improvement suggestions** — telling users specifically what would make their password stronger.
4. **Estimated crack time** — showing how long a brute-force attack would take to crack the current password.
5. **Breached credential checking** — verifying whether a password appears in known breach databases using k-Anonymity.
6. **User registration and authentication** — allowing users to create accounts and log in securely.
7. **Check history** — saving breach check results for authenticated users to review later.

2.3 User Classes

User Class Description

Guest User: A visitor who has not registered or logged in. Can use the strength meter and breach checker but cannot save history. **Registered User:** A user with an account who is logged in. Has full access to all features including check history.

Administrator: Not in scope for this version. Future versions may include admin functionality.

2.4 Operating Environment

- **Client side:** Any modern web browser (Chrome, Firefox, Safari, Edge) on desktop or mobile.
- **Server side:** Node.js v22 runtime hosted on Render (cloud platform).
- **Database:** MongoDB Atlas (cloud-hosted, M0 free tier).
- **Frontend hosting:** Vercel (cloud platform).
- **Minimum screen size supported:** 320px width (small mobile).
- **Internet connection:** Required for breach checking and authentication features.

2.5 Assumptions and Dependencies

Assumptions or Dependencies

- 1 The Have I Been Pwned API remains publicly available and free to use
- 2 Users have a modern browser with JavaScript enabled
- 3 MongoDB Atlas free tier remains available during the project lifecycle
- 4 The zxcvbn library accurately estimates password strength
- 5 Users understand that the breach checker cannot detect breaches that have not yet been publicly disclosed

3. Functional Requirements

3.1 Password Strength Analysis

ID - Requirement - Priority

FR-01 - The system shall analyse password strength in real time as the user types, with no more than 100ms delay - High . FR-02 - The system shall display a visual strength indicator using a colour-coded progress bar - High . FR-03 - The system shall display a text label indicating strength level: Weak, Fair, Good, Strong, or Very Strong - High . FR-04 - The system shall display specific feedback messages explaining why a password scored at its current level - High . FR-05 - The system shall display an estimated time required to crack the password using brute force - Medium . FR-06 - The system shall allow the user to toggle password visibility (show/hide) - Medium .

3.2 Breached Credential Checking

ID - Requirement - Priority

FR-07 - The system shall allow the user to check whether their password appears in known breach databases - High FR-08 - The system shall implement k-Anonymity, only the first 5 characters of the SHA-1 hash shall be sent to the HIBP API - High. FR-09 - The system shall display how many times the password was found in breaches, if any - High . FR-10 - The system shall display a clear safe/unsafe result message after each breach check - High.

3.3 User Authentication

ID - Requirement - Priority

FR-11 - The system shall allow new users to register with a username, email, and password - High . FR-12 - The system shall allow registered users to log in using their email and password - High . FR-13 - The system shall issue a JWT token upon successful login - High. FR-14 - The system shall allow users to log out, invalidating their session - High. FR-15 - The system shall protect authenticated routes from unauthenticated access - High.

3.4 Check History

ID - Requirement - Priority

FR-16 - The system shall save breach check results for authenticated users - Medium. FR-17 - The system shall display a history page showing past checks with timestamps - Medium. FR-18 - The system shall NOT store the actual password — only a masked version and the result - High .

3.5 Responsive Design

ID - Requirement - Priority

FR-19 - The system shall display correctly on screen widths from 320px to 1920px High. FR-20 - The system shall adapt its layout for mobile, tablet, and desktop viewports - High.

4. Non-Functional Requirements

4.1 Performance

ID - Requirement

NFR-01 - Password strength analysis shall respond within 100 milliseconds of user input. NFR-02 - The application shall load within 3 seconds on a standard broadband connection. NFR-03 - API responses shall be returned within 2 seconds under normal conditions.

4.2 Security

ID - Requirement

NFR-04 - No raw password shall ever be transmitted to any external server. NFR-05 - All user passwords shall be hashed using bcrypt with a minimum salt round of 10 before storage. NFR-06 - JWT tokens shall expire after 7 days. NFR-07 - All API endpoints shall implement input validation and sanitisation. NFR-08 - The backend API shall implement rate limiting to prevent brute force and abuse. NFR-09 - All production communication shall occur over HTTPS.

4.3 Usability

ID - Requirement

NFR-10 - The interface shall be usable by a non-technical user without any instructions. NFR-11 - Strength feedback shall be written in plain language, not technical jargon. NFR-12 - Error messages shall clearly explain what went wrong and what the user should do.

4.4 Maintainability

ID - Requirement

NFR-13 - The codebase shall follow a clear separation of concerns between frontend, backend, and database layers. NFR-14 - All environment-specific configuration shall be stored in environment variables, never hardcoded. NFR-15 - The project shall be version controlled using Git with descriptive commit messages.

5. Use Case Analysis

5.1 Actors

- **Guest User** — an unauthenticated visitor
- **Registered User** — an authenticated user

- **HIBP API** — external breach database service
- **MongoDB Atlas** — external database service

5.2 Use Cases

UC-01: Check Password Strength

Actor: Guest User or Registered User

Precondition: User has the application open in a browser

Main Flow:

1. User types a password into the input field
2. System analyses the password using zxcvbn
3. System updates the strength bar, label, and feedback in real time
4. User reads the feedback and adjusts their password if needed

Postcondition: User sees a strength score and improvement suggestions

UC-02: Check Breached Credentials

Actor: Guest User or Registered User

Precondition: User has entered a password

Main Flow:

1. User clicks the “Check for Breaches” button
2. System generates a SHA-1 hash of the password
3. System sends the first 5 characters of the hash to the HIBP API
4. HIBP API returns a list of matching hash suffixes and breach counts
5. System checks locally whether the full hash appears in the response
6. System displays the result to the user

Postcondition: User sees whether their password was found in known breaches

Alternative Flow: If the HIBP API is unavailable, the system displays an appropriate error message

UC-03: Register an Account

Actor: Guest User

Precondition: User does not have an existing account

Main Flow:

1. User navigates to the Register page
2. User enters a username, email, and password
3. System validates the input fields
4. System hashes the password using bcrypt
5. System saves the new user to MongoDB
6. System issues a JWT token and logs the user in

Postcondition: User has an account and is authenticated

UC-04: Log In

Actor: Registered User

Precondition: User has an existing account

Main Flow:

1. User navigates to the Login page
2. User enters their email and password
3. System retrieves the user record from MongoDB
4. System compares the entered password against the stored hash using bcrypt
5. System issues a JWT token on success

Postcondition: User is authenticated and can access protected features

Alternative Flow: If credentials are wrong, system displays an error without revealing which field was incorrect

UC-05: View Check History

Actor: Registered User

Precondition: User is logged in and has performed at least one breach check

Main Flow:

1. User navigates to the History page
2. System retrieves the user's check history from MongoDB
3. System displays a list of past checks with timestamps and results

Postcondition: User sees their breach check history

6. System Architecture

6.1 Architecture Overview

PassGuard follows a three-tier client-server architecture:

- **Presentation Tier:** React.js frontend hosted on Vercel
- **Application Tier:** Node.js and Express.js REST API hosted on Render
- **Data Tier:** MongoDB Atlas cloud database

6.2 Architecture Description

The user interacts with the React frontend in their browser. The frontend communicates with the Express backend via HTTP requests. The backend processes requests, applies business logic, communicates with MongoDB for data storage, and communicates with the HIBP API for breach detection.

Passwords never leave the browser in raw form. SHA-1 hashing occurs client-side before any network request is made for breach checking.

6.3 Technology Stack

Layer - Technology - Purpose

Frontend - React.js - UI components and user interaction Styling - Tailwind CSS - Responsive design system Backend - Node.js + Express.js - REST API and business logic Database - MongoDB + Mongoose - Data storage and modelling Authentication - JWT + bcrypt - Secure user sessions Strength Analysis - zxcvbn - Password scoring algorithm Breach Detection - HIBP API - Known breach database Frontend Hosting - Vercel - Static frontend deployment Backend Hosting - Render - Node.js API deployment Version Control - Git + GitHub - Source code management

7. Database Design

7.1 Collections

PassGuard uses two MongoDB collections.

Collection: users

Field - Type - Description

_id - ObjectId - Auto-generated unique identifier username - String - The user's chosen display name email - String - Unique email address used for login password - String - bcrypt-hashed password — never plain text createdAt - Date - Account creation timestamp

Collection: checkhistory

Field - Type - Description

_id - ObjectId - Auto-generated unique identifier userId - ObjectId - Reference to the users collection maskedPassword - String - Password with most characters hidden e.g. p*****rd breachFound - Boolean - Whether the password was found in a breach breachCount - Number - How many times it appeared in breaches strengthScore - Number - zxcvbn score from 0 to 4 checkedAt - Date - Timestamp of the check

7.2 Design Decisions

- Actual passwords are never stored in any form
- The maskedPassword field stores only a visual representation for history display
- userId links history records to users without embedding — keeping collections lean

8. API Overview

All API routes are prefixed with /api.

8.1 Authentication Routes

Method - Endpoint - Description - Auth Required

POST - /api/auth/register - Register a new user - No POST - /api/auth/login - Log in and receive JWT - No

8.2 Password Routes

Method - Endpoint - Description - Auth Required

POST - /api/password/check-breach - Check password against HIBP - No

8.3 History Routes

Method - Endpoint - Description - Auth Required

GET - /api/history - Get check history for logged-in user - Yes DELETE -

/api/history/:id - Delete a specific history record - Yes

9. Security Considerations

Concern - Mitigation

Password storage - bcrypt hashing with salt rounds of 10 — passwords are never stored in readable form Raw password transmission - SHA-1 hashing occurs in the browser — no raw password is ever sent over the network API abuse - Rate limiting applied to all API endpoints using express-rate-limit Token security JWT tokens expire after 7 days and are stored in httpOnly cookies or localStorage with XSS awareness Input attacks - All user inputs are validated and sanitised before processing Environment secrets - All credentials stored in .env files excluded from version control via .gitignore HTTPS - All production traffic served over HTTPS via Vercel and Render

10. Testing Strategy

Test Type - What Is Tested - Tool

Manual functional testing - All user-facing features work as expected - Browser + Postman API testing - All endpoints return correct responses and status codes - Postman Responsive testing - Application displays correctly on mobile and desktop - Chrome DevTools Security testing - Breach checker never sends raw passwords - Network tab in DevTools Edge case testing - Empty input, very long passwords, special characters - Manual testing